6.2 ESP32-Cam Library File Introduction

This section explains the program analysis of the “hw_esp32cam _ctl.cpp” library file.

1. Getting Ready

1) Open any program in the path of the “1.Tutorials\6.AI Vision Game Course”. Take the color sorting as an example.

actions.h	2024/3/21 12:29	H 文件	1 KB
hw_esp32cam_ctl.cpp	2024/3/21 12:29	CPP 文件	3 KB
hw_esp32cam_ctl.h	2024/3/21 12:29	H 文件	1 KB
mini_servo.cpp	2024/3/21 12:29	CPP 文件	3 KB
mini_servo.h	2024/3/21 12:29	H 文件	1 KB
MiniArm_color_clamp.ino	2024/3/21 18:25	INO 文件	6 KB
tone.h	2024/3/21 12:29	H 文件	6 KB

2) In the program, select “hw_esp32cam_ctl.cpp”.

2. Brief Program Analysis

1) Initialize the function of I2C communication: “begin()”.

```
void HW_ESP32Cam::begin(void)
{
  Wire.begin();
}
```

2) Send multiple bytes to esp32Cam: “wireWriteDataArray()”.

```
//写多个字节
static bool wireWriteDataArray(uint8_t addr, uint8_t reg,uint8_t *val,unsigned int len)
{
    unsigned int i;

    Wire.beginTransmission(addr);
    Wire.write(reg);
    for(i = 0; i < len; i++)
    {
        Wire.write(val[i]);
    }
    if( Wire.endTransmission() != 0 )
    {
        return false;
    }
    return true;
}
```

Parameters:

addr: esp32Cam device address（0x52）

reg: the target memory address for writing to esp32Cam

*val: data address to be sent

len: word length to be sent

The master Arduino sends the device address via I2C first and the target memory address to esp32Cam. Then it sends the data by bytes according to the data length. Next, it checks if the data has been successfully transmitted using “endTransmission()”. If the return value is 0, it indicates successful data transmission; otherwise, it is fail to send.

3) Send one byte to esp32Cam: “wireWriteByte()”.

```
static bool WireWriteByte(uint8_t val)
{
    Wire.beginTransmission(ESP32CAM_ADDR);
    Wire.write(val);
    if( Wire.endTransmission() != 0 ) {
        return false;
    }
    return true;
}
```

Parameter:

val: content of the sent data

The Arduino master directly sends the device address and the data content of the 1 byte to esp32Cam. The “endTransmission()” checks the data sent successfully or not. If the return value is 0, the data is sent successfully.

Otherwise it is sent unsuccessfully.

4) Read the data of multiple bytes from esp32Cam: “wireReadData Array()”.

```
static int WireReadDataArray(uint8_t reg, uint8_t *val, unsigned int len)
{
    unsigned char i = 0;

    /* Indicate which register we want to read from */
    if (!WireWriteByte(reg)) {
        return -1;
    }

    /* Read block data */
    Wire.requestFrom(ESP32CAM_ADDR, len);
    while (Wire.available()) {
        if (i >= len) {
            return -1;
        }
        val[i] = Wire.read();
        i++;
    }
    return i;
}
```

Parameters:

reg: the address of the data to be read in esp32Cam

*val: the address used for saving the data in Arduino master

len: Length of data to be read

The Arduino master first sends the address of the data to be read to esp32Cam. If there is no response, it returns -1, indicating a failed connection. Then, it sends the length of data to esp32Cam, and reads the data in bytes via the “while” loop and the “i” variables to store in the “val[]” array. During the reading process, if the actual read length exceeds the length to be read, it returns to -1, indicating a reading failure. Finally, upon successful reading, it returns “i”, which represents the actual length of read data (where “i” may be less than or equal to “len”).

5) Read the result of esp32Cam face detection: **“faceDetect()”**

```
//Reading ESP32Cam to detect faces
bool HW_ESP32Cam::faceDetect(void)
{
  uint8_t face_info[4];
  Serial.print("face ");
  int num = WireReadDataArray(0x01,face_info,4);
  if((num == 4) && (face_info[2] > 0))
  //Received the x, y, w, h values of the recognized face
  {
      Serial.println(" 1");
      return true;
  }
  Serial.println(" 0");
  return false;
}
```

Define the “face_info[4]” array to save the data. From 0x01address, read four bytes to save into the “face_info[4]” array through “wireReadDataArray()” function. The meaning of each element is as follows:

face_info[0] : x (the x-coordinate value identified at the upper-left corner of the face region)

face_info[1]: y (the y-coordinate value identified at the upper-left corner of the face region)

face_info[2] : w (the identified width of the face region)

face_info[3]: h (the identified height of the face region)

Receive the length of the data by “num” variable. If it is 4 and “face_info[2]”(the value of w) is more than 0, the face is identified to return ‘true’. Or return to ‘false’.

6) Read the result of esp32Cam color reorganization: “colorDetect()”

```
/Read ESP32Cam to recognize colors and return color codes
nt HW_ESP32Cam::colorDetect(void)

 uint8_t color_info[2][4];
 int num = WireReadDataArray(0x00,color_info[0],4);
 if((num == 4) && (color_info[0][2] > 0))
 //Receive the x, y, w, h values of the recognized colors
 {
     return 1;
 }
 num = WireReadDataArray(0x01,color_info[1],4);
 if(num == 4)
 {
   if(color_info[1][2] > 0)
   //If the value of w is greater than 0, color 1 is recognized
   {
     return 2;
   }
 }
 return 0;
```

Define “color_info[2][4]” bi-dimensional array to save the data, the meaning of each element is as below:

color_info[*][0] : x (the x-coordinate value identified at the upper-left corner of the color region)

color_info[*][1]: y (the y-coordinate value identified at the upper-left corner of the color region)

color_info[*][2]: w (the identified width of the color region)

color_info[*][3]: h (the identified height of the color region)

So if * is 0, the recognition color is 1. If * is 1, the recognition color is 2.

Using the “wireReadDataArray()” function, read four bytes of data from the address 0x00. These bytes represent the parameter information for color 1 detection and are stored in the “color_info[0][4]” array. Determine whether color 1 is detected based on the data length received by num and the value of “color_info[0][4]” (i.e., the width of the region where color 1 is detected). If num

equals 4 and “color_info[0][2]” is greater than 0, color 1 is detected, and the function returns 1.

If color 1 is not detected, according to the above steps, read data from the address 0x01 to store in the “color_info[1][4]” array, and then determine whether color 2 is detected. If it is detected, the function returns 2.

If none of the above results, the function returns to 0.

7) Read the position of esp32Cam color recognition: “color_position()”

```
/* Read ESP32Cam to recognize color position,
successfully read and return true and position data */
bool HW_ESP32Cam::color_position(uint16_t *color_info)
{
  int num = WireReadDataArray(0x01,(uint8_t*)color_info,8);
  if((num == 8) && (color_info[2] > 0))
  //Receive the x, y, w, h values of the recognized colors
  {
    return true;
  }
  return false;
}
```

Parameter and meaning are as below:

*color_info： save the address of the received data

By the “wireReadDataArray ()” function, eight bytes of data read from the address of 0x01 are stored in the address showed by “color_info[0][4]”. Judge whether recognizing the color or not according to the data length received by “num” and the value of “color_info[2]” (the recognized width of the color region). If “num” equals 8 and “color_info[2]” is greater than 0, it recognizes color to return to “true”. Or return to “false”.

8) Control the brightness of esp32Cam’s fill lights: “set_led()”

```
void HW_ESP32Cam::set_led(uint8_t lightness)
{
  lightness = lightness < 0 ? 0 : lightness;
  lightness = lightness > 255 ? 255 : lightness;
  wireWriteDataArray(ESP32CAM_ADDR , 0x11 , &lightness , 1);
}
```

Parameter is as below:

lightness：the brightness of the fill lights（0~255）

Firstly, check if the received parameter is within bounds. This is done by verifying if the parameter value falls within the range of 0 to 255. If the value is less than 0, it is set to 0; if it exceeds 255, it is capped at 255. This ensures the validity of the incoming parameter. Finally, use the “wireWriteDataArray()” function to send the lightness parameter value to address 0x11 on the ESP32-Cam.